CureHub – Hospital Management System

A Centralized Multi-Hospital Platform for Government Hospitals

API & Microservices

A Cornerstone Project Report submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

Desina Rohit	(24B11AI084)
Surendra Kumar Battagani	(24B11AI031)
Gowthu Manikanta Saran Teja	(24B11AI129)
Chakradhar Chowdary Gunnam	(24B11AI054)

Under the Guidance of

Mrs. Khasimbee Shaik, M. Tech, (Ph.D)
Assistant Professor, AIML

ADITYA UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

2024-2028

ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work entitled **"CureHub – Hospital Management System"** is submitted by

Desina Rohit	(24B11AI084)
Surendra Kumar Battagani	(24B11AI031)
Gowthu Manikanta Saran Teja	(24B11AI129)
Chakradhar Chowdary Gunnam	(24B11AI054)

in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial Intelligence and Machine Learning for the academic year 2026-2027.

Project Guide
Mrs. Khasimbee Shaik, M.Tech, (Ph.D).
Assistant Professor, AIML

Head of the Department
Dr. K. Naga Bhargavi, M.Tech, Ph.D.
Associate Professor & HOD, AIML

External Examiner

DECLARATION

We hereby declare that the project entitled **"CureHub – Hospital Management System"** is original and is submitted to **ADITYA UNIVERSITY, Surampalem** in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial Intelligence and Machine Learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date: 14/10/2026

By
Desina Rohit (24B11AI084)
Surendra Kumar Battagani (24B11AI031)
Gowthu Manikanta Saran Teja (24B11AI129)
Chakradhar Chowdary Gunnam (24B11AI054)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and faculty members for their guidance during the development of the E-Commerce Product and Order Management System. Their support helped us connect API design, persistence and microservice communication with a working application.

We thank the Department of Artificial Intelligence and Machine Learning and the institution for providing the learning environment and resources required for this academic project. The coursework in API and Microservices provided the foundation for implementing REST services, SOAP integration, validation, transaction management and aspect-oriented logging.

We also thank our classmates and everyone who supported the discussions, demonstrations and review of the project. This work strengthened our understanding of service boundaries, reliable checkout processing and the responsibilities of customers, sellers and administrators in an e-commerce application.

ABSTRACT

The E-Commerce Product and Order Management System is an academic web application that supports product browsing, seller listings, customer checkout, order tracking and invoice generation. It combines an Angular storefront with independently executable Java Spring Boot services. An API Gateway provides a common entry point, while User, Product, Order and Invoice Services own their respective application data.

The backend targets Java 21 and uses Spring Boot 3.5.16, Spring Data JPA, Spring Security, Bean Validation and Spring AOP. Four logical CockroachDB databases store accounts, catalog and inventory records, orders and recovery workflows, and immutable invoice snapshots. Flyway maintains versioned schemas. REST endpoints serve the storefront, and the Order Service invokes a contract-first SOAP service to generate order bills.

Each purchase uses an idempotency key, a persistent workflow and repeatable inventory operations. Stock reservations protect availability, confirmed orders retain purchase-price snapshots, and cancellation restores stock once. A recovery worker resumes interrupted requests. JWT authentication and server-side role and ownership checks distinguish customer, seller and administrator access.

The saved verification record reports 23 backend integration tests, live gateway and CockroachDB checks, frontend regression checks and a successful frontend build. Payment selections are demonstration choices and do not collect money. The project illustrates how REST, SOAP and local database transactions can cooperate in a recoverable e-commerce workflow without claiming a distributed atomic transaction.

Keywords: E-Commerce, Microservices, Spring Boot, Angular, REST API, SOAP, CockroachDB, Spring Data JPA, JWT, Idempotency, Inventory Reservation, Order Recovery

TABLE OF CONTENTS

Section / Chapter	Title	Page
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives of the Project	1
1.4	Scope of the Project	2
2	SYSTEM OVERVIEW	3
2.1	System Architecture	3
2.2	System Flow	3
2.3	User Roles	4
2.4	System Modules	4
3	TECHNOLOGIES USED	5
3.1	Java	5
3.2	Spring Boot	5
3.3	Angular and TypeScript	5
3.4	RESTful Web Services	6
3.5	Spring Data JPA	6
3.6	CockroachDB and H2	6
3.7	Spring AOP	6
3.8	Bean Validation and Exception Handling	6
3.9	SOAP Web Services	6
3.10	Security and CORS	7
3.11	Maven and Testing Tools	7
4	DATABASE DESIGN	8
4.1	Database Tables Overview	8
4.2	Users and User Roles Tables	8
4.3	Products Table	9
4.4	Stock Reservations Tables	9
4.5	Orders Table	9

Section / Chapter	Title	Page
4.6	**Order Items Table**	10
4.7	**Order Workflows Table**	10
4.8	**Invoices Table**	10
4.9	**Invoice Items Table**	10
4.10	**Database Relationships and Integrity**	10
5	**SYSTEM MODULES**	11
5.1	**User Registration and Authentication**	11
5.2	**Product Catalog**	11
5.3	**Seller Product Management**	11
5.4	**Product Image Management**	11
5.5	**Cart and Checkout**	11
5.6	**Inventory and Recovery**	11
5.7	**Cancellation**	12
5.8	**SOAP Invoice Generation**	12
5.9	**Administrator Portal**	12
5.10	**Logging and Demonstration Data**	12
6	**API AND IMPLEMENTATION**	13
6.1	**Frontend Implementation**	13
6.2	**Backend Implementation**	13
6.3	**REST API Operations**	13
6.4	**Transaction Management**	14
6.5	**Spring AOP Logging**	15
6.6	**Validation and Exception Handling**	15
6.7	**SOAP Web Service**	16
6.8	**Security and Pagination**	16
6.9	**Testing**	16
6.10	**Running the Project**	17
7	**OUTPUT SCREENS AND RESULTS**	18
7.1	**Desktop Storefront**	18

Section / Chapter	Title	Page
7.2	**Mobile Storefront**	19
7.3	**Customer and Seller Workflows**	19
7.4	**Checkout and Inventory Results**	19
7.5	**Invoice and Cancellation Results**	20
7.6	**Validation and Access Results**	20
7.7	**Automated Test and Build Results**	20
7.8	**Observations and Limitations**	21
8	**CONCLUSION**	22
8.1	**Summary**	22
8.2	**Future Enhancements**	22

LIST OF FIGURES

Figure	Description	Page
Figure 2.1	System Architecture	3
Figure 4.1	Database Relationships	8
Figure 7.1	Desktop Storefront	18
Figure 7.2	Mobile Storefront	19
Figure 7.3	Recoverable Checkout Workflow	20

CHAPTER 1
INTRODUCTION

1.1 Introduction

An online storefront must coordinate information about users, products, inventory and orders. Browsing a product is a read operation, but placing an order changes persistent state and involves several business rules. The server must identify the buyer, obtain the current price, check stock, record the purchase and preserve enough information to recover from a temporary failure.

This project implements those requirements through four business services and an API Gateway. The Angular application presents the catalog and shopping flows. The User Service manages authentication and roles, the Product Service manages listings and inventory, the Order Service coordinates purchases, and the Invoice Service produces order bills through SOAP. Each service has its own logical database in a CockroachDB cluster.

The implementation demonstrates constructor dependency injection, REST resource handling, request validation, JPA repositories, transaction boundaries, AOP logging, exception handling, authentication, pagination and service-to-service communication. It also addresses duplicate requests and interrupted workflows, which cannot be solved by interface validation alone.

1.2 Problem Statement

- Product and stock records must remain consistent when several customers purchase the same item concurrently.
- A repeated checkout request after a timeout must not create a second order or deduct inventory twice.
- A failure between stock confirmation and order confirmation must leave durable information to resume the purchase.
- Sellers must manage only their own listings, while customers must access only their own order history.
- Invoices must preserve original customer, product and price information when listings change later.
- The application must expose clear REST responses and integrate SOAP billing without exposing internal credentials to the browser.

1.3 Objectives of the Project

- Develop an Angular storefront for catalog search, filtering, cart management, checkout and order history.

- Implement independent User, Product, Order and Invoice Services behind an API Gateway.
- Enforce JWT authentication, role permissions and record ownership in backend services.
- Maintain inventory through atomic reservations, confirmation and cancellation operations.
- Provide idempotent checkout and durable recovery for interrupted cross-service work.
- Generate immutable order bills through a contract-first SOAP interface.
- Apply Bean Validation, consistent errors, AOP logging and automated verification.

1.4 Scope of the Project

The implemented scope includes registration, customer and seller login, dedicated administrator login, seller enablement, catalog search, product ownership, image uploads, inventory management, structured delivery details, order placement, cancellation and invoice retrieval. Administrator endpoints provide customer lookup, order browsing, order counts and customer order statistics. The storefront includes desktop and mobile layouts.

Cash on delivery and online payment are stored demonstration choices with the displayed status DEMO_NOT_COLLECTED; no payment gateway collects money. Delivery fulfilment, tax calculation, statutory seller invoicing, invoice PDF generation, credit notes and refunds are outside the implemented scope. A reachable CockroachDB cluster and preserved local product-image storage are required for the configured deployment.

CHAPTER 2
SYSTEM OVERVIEW

2.1 System Architecture

The API Gateway listens on port 8080 and routes public requests to the business services. Angular runs on port 4200 during development. User, Product, Order and Invoice Services listen on ports 8081, 8082, 8083 and 8084 respectively. The common Maven module supplies reusable security, validation, error, logging and transaction support; it is a library rather than another running service.

Order Service communicates with Product Service over internal REST endpoints and with Invoice Service over SOAP. These calls use an internal service key and are not exposed as gateway routes. Business services independently validate JWTs and authorization. Separate logical databases limit persistence responsibilities, while cross-service identifiers are stored without cross-database foreign keys.

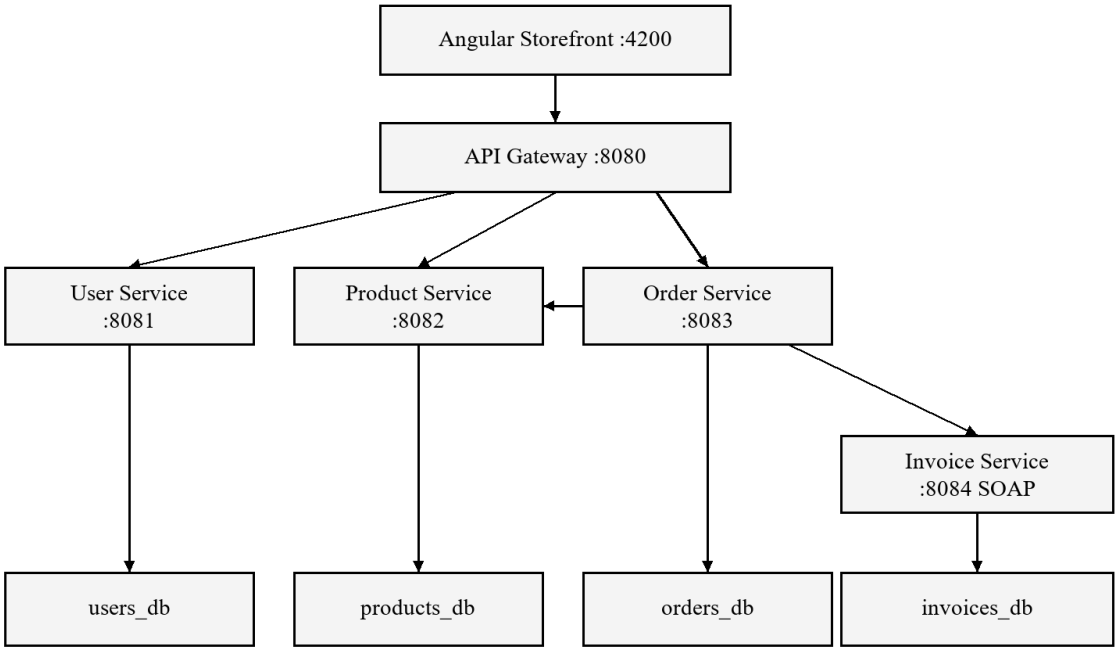


Figure 2.1: System Architecture

2.2 System Flow

- A visitor browses active products with category, price, search and availability filters.
- The customer registers or logs in and receives a signed JWT for authenticated requests.
- The customer adds cart items, enters delivery details and selects a demonstration payment method.

- Checkout sends an idempotency key. Order Service persists a PENDING order and workflow.
- Product Service reserves all requested items atomically and returns authoritative prices and names.
- Order Service saves the snapshot, confirms inventory and marks the order CONFIRMED. Interrupted work remains recoverable.
- The customer views the order, cancels an eligible purchase or requests a SOAP-generated invoice.
- Sellers manage owned listings, while administrators manage the complete catalog and inspect platform orders.

2.3 User Roles

Authorization is enforced by the server as well as reflected in the interface. Knowing an identifier does not grant permission to change or read a protected record.

Table 2.1: User Roles

Role	Main Access	Responsibilities
Visitor	Public catalog	Browse products; register or sign in.
Customer	Own account and orders	Place purchases, view own orders and request eligible cancellation or billing.
Seller	Owned listings	Manage product details, photos and stock.
Administrator	Administrative APIs	Manage any listing; inspect customers, orders and summary counts.
Internal service	Protected internal routes	Reserve stock and request invoices using a service credential.

2.4 System Modules

- User management: credentials, profiles, seller enablement and administrator authentication.
- Catalog management: product details, photos, inventory and server-side filtering.
- Order management: checkout, snapshots, buyer history, recovery and cancellation.
- Invoice management: SOAP validation and immutable billing records.
- Gateway and common infrastructure: routing, CORS, security, errors, logging and retries.
- Angular frontend: browsing, customer, seller and administrator workflows.

CHAPTER 3
TECHNOLOGIES USED

3.1 Java

Backend source targets Java 21. Classes and records represent entities, requests and responses, while typed service methods implement product, order and billing rules. Monetary values use decimal representations rather than binary floating-point calculations. The project documentation records JDK 25 as the tested local Java environment.

3.2 Spring Boot

Spring Boot 3.5.16 manages configuration, dependency injection and embedded execution. The Maven project builds a shared library and five applications. Each service has configuration for its port, database and dependencies. Constructor injection makes dependencies explicit and allows tests to substitute controlled clients for remote failure scenarios.

Table 3.1: Backend Dependencies

Library or Framework	Purpose
Spring Boot 3.5.16	Configuration and executable services.
Spring Cloud 2025.0.1	Gateway dependencies and routing.
Spring Data JPA and Hibernate	Entity mapping and repositories.
Spring Security and JWT	Authentication and role checks.
Spring Web Services	Contract-first SOAP invoicing.
Bean Validation and Spring AOP	Request constraints and operation logging.
Flyway 13.9.0	Versioned database migrations.
H2 and Spring testing support	Isolated integration tests.

3.3 Angular and TypeScript

The frontend package specifies Angular 22.2 and TypeScript 6.0. It implements catalog views, account flows, cart and checkout, seller listing forms and an administrator portal. Angular services centralize HTTP requests, and an authentication interceptor attaches bearer tokens to the project API. The development server runs on port 4200.

Cart validation checks stored quantities, stock limits, prices and unavailable listings. Interrupted checkout retains the original request key and frozen payload in session storage. The same payload is reused after a timeout even when the visible cart changes, preventing a retry from becoming a different purchase under the same identity.

3.4 RESTful Web Services

REST endpoints accept and return JSON through HTTP. GET reads resources, POST creates records or initiates actions, PUT edits listing details, PATCH sets stock and DELETE deactivates a product. Status codes distinguish successful operations, validation failures, denied access, missing records and conflicts. PENDING orders return 202 with a location that clients can poll.

3.5 Spring Data JPA

JPA entities model users, products, reservations, orders, workflows and invoices. Repositories provide persistence operations, and specifications filter data before pagination. Hibernate validates the schema, while Flyway migrations create and evolve it. Local associations use foreign keys; identities owned by another service remain stored UUID values.

3.6 CockroachDB and H2

The application uses users_db, products_db, orders_db and invoices_db in a shared CockroachDB cluster. JDBC connections require certificate and hostname verification. The supplied deployment currently uses a shared database login; separate logins would strengthen database-level isolation. H2 is used by automated integration tests and is not the configured live persistence system.

3.7 Spring AOP

The common OperationLogging aspect records the operation, elapsed time, outcome and trace identifier. Centralizing this behaviour avoids repetitive logging in business methods. Request payloads are not included in the operation log, helping keep credentials and delivery details out of routine diagnostic output.

3.8 Bean Validation and Exception Handling

Bean Validation checks required fields, lengths and numeric ranges. Business validation checks ownership, available stock, compatible idempotency keys and permitted state transitions. Shared REST error handling returns code, message, fields and traceId. SOAP validation produces faults suitable for the XML client.

3.9 SOAP Web Services

Invoice Service exposes SOAP 1.1 at /ws and a WSDL at /ws/invoices.wsdl on port 8084. The XML namespace is urn:ecom:invoice:v1. The contract defines generateInvoiceRequest and generateInvoiceResponse. Order Service sends a purchase snapshot and receives a durable invoice; identical retries return the same invoice identity.

3.10 Security and CORS

Passwords use BCrypt hashes. JWTs carry roles derived from saved records and expire after thirty minutes. Public registration cannot create an administrator. Services check issuer, signature, expiry, role and ownership. The gateway restricts browser origins using FRONTEND_ORIGIN and strips client-supplied internal service keys. Internal credentials are used only by trusted backend callers.

3.11 Maven and Testing Tools

Maven builds the backend and runs integration tests. Spring testing support and H2 exercise persistence and authorization, while HTTP SOAP tests verify the invoice contract. PowerShell scripts start the applications and perform live checks. Frontend regression scripts and the Angular build verify client behaviour. Postman definitions support academic demonstrations.

CHAPTER 4
DATABASE DESIGN

4.1 Database Tables Overview

The core schema contains ten domain tables across four databases, excluding Flyway history tables. Users and user_roles belong to User Service; products, stock_reservations and stock_reservation_items belong to Product Service; orders, order_items and order_workflows belong to Order Service; invoices and invoice_items belong to Invoice Service. This separation keeps business ownership explicit.

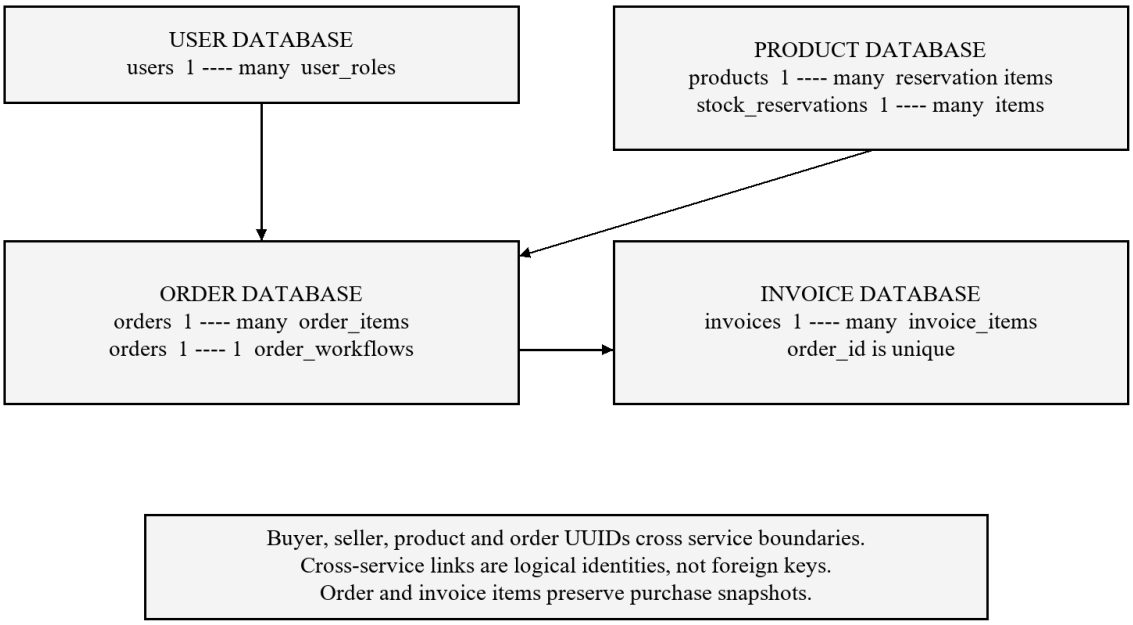


Figure 4.1: Database Relationships

4.2 Users and User Roles Tables

The users table stores account identity and authentication data. The user_roles table stores permitted roles. A local foreign key links roles to users, and the composite primary key prevents duplicate assignment of a role.

- users.id: UUID primary key; email: required and unique.
- name, password_hash, enabled and created_at: profile, password hash, account state and timestamp.
- user_roles.user_id and role: composite key; permitted values are CUSTOMER, SELLER and ADMIN.

4.3 Products Table

The products table stores catalog content and inventory counters together. seller_id identifies the owner in User Service without a cross-database foreign key. Deactivation preserves the record for historical purchases.

- id: UUID primary key; sku: unique product code; seller_id: owner identity.
- name, description, category, image_url and active: visible fields and listing state.
- price: DECIMAL(19,2), greater than zero; currency: three-character code.
- stock_on_hand and reserved: nonnegative integers with reserved no greater than stock_on_hand.
- version and created_at: concurrency metadata and creation time; indexes support seller and catalog queries.

4.4 Stock Reservations Tables

stock_reservations uses order_id as its primary key so repeated inventory calls address the same reservation. request_hash identifies the original item request. Status is RESERVED, CONFIRMED, RELEASED or CANCELLED. stock_reservation_items stores the product, seller, name, unit price and quantity snapshot for each line.

- Reservation items reference their reservation and product within products_db.
- The primary key is order_id plus line_no; a unique constraint prevents duplicate product lines.
- Positive quantity constraints and transactional stock checks prevent invalid or partially reserved purchases.

4.5 Orders Table

The orders table records the buyer, contact details, delivery information, totals and state. The unique pair buyer_id and idempotency_key prevents a second order for the same buyer request identity. request_hash detects changed content under an existing key.

- id and buyer_id: order and purchasing account identities.
- status: PENDING, CONFIRMED, FAILED, CANCEL_PENDING or CANCELLED.
- customer_name, customer_email, address and structured delivery columns: saved contact and destination details.
- currency and total: monetary snapshot; total uses DECIMAL(24,2).
- invoice_requested, invoice_xml, version and created_at: billing guard, cached response, concurrency and creation data.

- payment_method: COD or ONLINE_DEMO, presented as unpaid demonstration choices.

4.6 Order Items Table

order_items stores each product and quantity with the purchase-time seller, name and unit price. Its primary key is order_id and line_no; a unique constraint covers order_id and product_id. Snapshot fields may remain null in the initial PENDING phase until the reservation response is saved. Confirmed purchases retain their values when the catalog is edited.

4.7 Order Workflows Table

order_workflows has one row per order and records step, retry_count, next_attempt, lease_until, lease_token and last_error. A foreign key links it to orders, and an index supports scanning eligible work. A durable lease coordinates the normal request path and recovery worker. Persistent workflow information makes process interruption recoverable without relying on an in-memory queue.

4.8 Invoices Table

invoices stores an immutable bill with a UUID primary key, unique invoice_number and unique order_id. Contact details, address, currency, total and issued_at preserve the billing snapshot. request_hash allows identical retries while rejecting changed billing data for an already invoiced order. The total must be positive.

4.9 Invoice Items Table

invoice_items stores invoice identity, line number, product and seller identities, name, quantity and unit price. Each row references invoices locally. Quantities and prices must be positive. Values are copied from the order snapshot, so later catalog changes cannot silently alter an issued invoice.

4.10 Database Relationships and Integrity

Local one-to-many relationships connect users to roles, reservations to items, orders to items and invoices to items. Orders have a one-to-one workflow. Product references in reservation items are local foreign keys. Buyer, seller, product and order references between databases are logical links maintained through service contracts.

Constraints, local serializable transactions and repeatable remote operations work together. Product rows are locked in stable ID order when reserving stock. Confirmation subtracts physical stock and releases reservations in one product transaction. Monetary snapshots preserve history, and unique identities make retries repeatable.

CHAPTER 5
SYSTEM MODULES

5.1 User Registration and Authentication

Registration creates a customer with a hashed password. Login returns a bearer token and profile. The profile endpoint requires authentication. Seller enablement adds the seller capability and returns a refreshed token. A dedicated administrator ID and password route authenticates the configured administrator; registration cannot grant this role.

5.2 Product Catalog

The public catalog exposes active listings and product details. Search, category, price, availability, sorting and pagination are applied by the backend, so totals describe all matching records. Sorting includes an identifier tiebreaker. The sample catalog contains 75 products across Electronics, Fashion, Appliances, Home and Books.

5.3 Seller Product Management

Sellers create, update and deactivate owned products. Product creation may include initial stock committed with the listing. A missing SKU is generated automatically. Updating details does not transfer ownership or silently change inventory. A separate stock endpoint sets current stock and rejects values below active reservations. Administrators may manage any listing.

5.4 Product Image Management

Seller and administrator forms accept JPG or PNG images up to 5 MB. Backend checks limit dimensions and total pixels, decode and re-encode valid images, and reject fake or unsupported files. Image bytes are stored locally while the database holds their public path. The uploads directory must accompany the data when the demonstration is moved.

5.5 Cart and Checkout

The cart validates selected quantities against current information. Checkout collects contact details and an Indian delivery address, including a six-digit pincode and ten-digit mobile number. The browser generates a request key and preserves the original payload for retries. The server obtains prices and seller identities from Product Service instead of trusting client totals.

5.6 Inventory and Recovery

Order Service persists the purchase and workflow before requesting inventory. Product Service reserves all items atomically or rejects the request. Order Service saves snapshots, confirms inventory and records confirmation. A worker resumes eligible workflows after interruption

using a lease and bounded retry delays. Dependency failures remain PENDING rather than being treated as completed purchases.

5.7 Cancellation

Cancellation compensates a confirmed purchase. Order Service records CANCEL_PENDING, Product Service restores inventory once, and the order becomes CANCELLED. Repeated calls use existing reservation state and do not repeatedly increase stock. Cancellation is prohibited after invoice generation starts, including while Invoice Service is unavailable.

5.8 SOAP Invoice Generation

Customers request billing through a REST order endpoint. Order Service authorizes the buyer, locks the eligibility decision and sets an invoice-request flag before invoking SOAP. Invoice Service validates the contract and persists one invoice per order. The result is returned as JSON to the storefront. Identical retries return the existing invoice.

5.9 Administrator Portal

Administrator endpoints support catalog management, non-administrator account lookup, order browsing by buyer and status, order summaries and selected customer order statistics. Backend role checks protect these operations. Pending summary counts include cancellation work that has not yet completed.

5.10 Logging and Demonstration Data

Shared logging records operation timing, outcome and trace identifiers. Consistent errors distinguish invalid fields, denied access, conflicts and dependency failures. Catalog seeding creates representative listings and skips existing SKUs on repeated imports. Verification scripts deactivate fixture products after successful checks, preserving order history without cluttering public browsing.

CHAPTER 6
API AND IMPLEMENTATION

6.1 Frontend Implementation

Angular routes requests through the gateway and presents customer, seller and administrator views. Catalog pages preserve backend search and pagination. Product forms support image selection. Customer views include delivery details, demonstration payment selection and order history. Authentication handling clears malformed or expired sessions, and seller enablement replaces the old token.

Session storage retains the checkout key and frozen request. Reopening the flow resumes that request instead of creating a second purchase. A confirmed checkout clears the cart, while a pending response can be polled. User-facing messages distinguish demonstration payment choices from actual collection.

6.2 Backend Implementation

Table 6.1: Backend Module Structure

Module	Responsibility
common	Security, errors, pagination, transactions and AOP.
api-gateway	Public routes, CORS and internal-key removal.
user-service	Accounts, authentication, roles and customer lookup.
product-service	Catalog, uploads and stock reservations.
order-service	Purchases, recovery, cancellation and invoice client.
invoice-service	SOAP validation and immutable invoice persistence.

6.3 REST API Operations

The public base address is http://localhost:8080. Protected requests include Authorization: Bearer followed by the access token. Internal inventory routes and SOAP are accessed directly by trusted services.

Table 6.2: REST API Operations

Method	Path	Purpose
POST	/api/auth/register	Register customer.
POST	/api/auth/login	Authenticate account.
POST	/api/auth/admin/login	Authenticate admin.
GET	/api/users/me	Read own profile.

Method	Path	Purpose
POST	/api/users/me/seller	Enable selling.
GET	/api/products	Browse catalog.
GET	/api/products/{id}	Read product.
GET, POST	/api/seller/products	List or create listings.
PUT, DELETE	/api/seller/products/{id}	Edit or deactivate.
PATCH	/api/seller/products/{id}/stock	Set stock.
POST	/api/seller/products/images	Upload photo.
GET, POST	/api/admin/products	Admin catalog.
POST, GET	/api/orders	Checkout or history.
GET	/api/orders/{id}	Read order.
POST	/api/orders/{id}/cancel	Cancel order.
POST, GET	/api/orders/{id}/invoice	Generate or read bill.
GET	/api/admin/users	Search customers.
GET	/api/admin/orders	Browse orders.
GET	/api/admin/orders/summary	Read counts.
POST	/api/admin/orders/customer-stats	Customer statistics.

The following illustrative checkout request uses placeholder credentials and product identity. Actual calls require a valid access token and product UUID.

```
POST /api/orders
Authorization: Bearer <access-token>
Idempotency-Key: checkout-example-001
Content-Type: application/json
{
  "customerName": "Demo Buyer",
  "customerEmail": "buyer@example.test",
  "address": "12 Demo Street, Kakinada",
  "deliveryAddress": {
    "line1": "12 Demo Street", "line2": "",
    "city": "Kakinada", "state": "Andhra Pradesh",
    "pincode": "533001", "country": "India",
    "phone": "9000000000"
  },
  "paymentMethod": "COD",
  "items": [{"productId": "<product-uuid>", "quantity": 2}]
}
```

6.4 Transaction Management

The shared Transactions helper uses TransactionTemplate with REQUIRES_NEW and SERIALIZABLE isolation. SQLSTATE 40001 conflicts are retried up to four times after the initial attempt with backoff and jitter. Each transaction applies to one database. HTTP and

SOAP calls occur outside these callbacks, so a remote timeout does not hold a local transaction open.

Checkout persists PENDING state, obtains a reservation, saves the snapshot, confirms inventory and confirms the order. A crash after inventory confirmation is recovered by reading the existing reservation. The workflow lease lasts ninety seconds. Recovery checks up to twenty workflows every fifteen seconds and bounds retry delays at three hundred seconds. This provides recoverable progress, not a single atomic commit across databases.

Table 6.3: Inventory State Transitions

Transition	Stock on Hand	Reserved
Reserve Q	Unchanged	Increase by Q
Confirm Q	Decrease by Q	Decrease by Q
Release unconfirmed Q	Unchanged	Decrease by Q
Cancel confirmed Q	Increase by Q once	Unchanged

6.5 Spring AOP Logging

OperationLogging intercepts service operations and records the operation name, duration, outcome and trace identifier. It provides diagnostic context without storing request bodies. Trace identifiers connect error responses with backend logs, and timing information helps identify slow operations. Logging remains separate from the main business rules.

6.6 Validation and Exception Handling

Product prices must be positive, inventory must respect reservations and checkout accepts one to fifty distinct products with bounded quantities. Reusing an idempotency key with different content is a conflict. Structured delivery validation checks required address fields, pincode and phone. Error responses give clients a stable structure for displaying failures.

Table 6.4: Exception to HTTP Status Mapping

Failure	Status	Behaviour
Invalid input	400	Field or request validation error.
Invalid authentication	401	Require a valid login.
Role or ownership denied	403	Reject the operation.
Missing resource	404	Report missing record.
Business conflict	409	Reject incompatible replay or state.

Failure	Status	Behaviour
Unexpected failure	500	Consistent error and trace identifier.
Invoice unavailable	503	Preserve guard and allow retry.

```
{
  "code": "VALIDATION_FAILED",
  "message": "Check the highlighted fields",
  "fields": {"price": "must be greater than or equal to 0.01"},
  "traceId": "<trace-uuid>"
}
```

6.7 SOAP Web Service

SOAP is published at http://localhost:8084/ws and its WSDL at /ws/invoices.wsdl. Both require X-Service-Key, which is never sent to the browser. The namespace urn:ecom:invoice:v1 identifies the contract for order identity, customer information, address, currency and item snapshots.

Table 6.5: SOAP Invoice Contract

Operation	Request	Response
generateInvoice	Order and billing snapshot	Invoice identity, total, issue time and items.
Identical retry	Same order and billing data	Original saved invoice.
Conflicting request	Invalid or changed payload	SOAP fault without replacing the bill.

The schema validates the request before invoice persistence. A unique order identity and request hash prevent duplicate or changed billing. Order Service translates the SOAP result into the JSON view used by the customer. No PDF or tax calculation is performed.

6.8 Security and Pagination

Catalog requests default to page zero and twelve results, with a maximum size of one hundred. Product sort fields are name, price and createdAt. Filtering occurs before pagination. Customer history remains scoped to the authenticated buyer; its PENDING filter includes CANCEL_PENDING work.

Each service validates JWTs. Registration cannot assign administrative authority, and seller modifications require ownership or an admin role. Logout removes the local token; already issued tokens are not immediately revoked. Asymmetric signing, separate service identities and revocation are potential production improvements.

6.9 Testing

The verification record dated 5 October 2026 reports the integration results below. Tests use isolated H2 databases with migrations. Order tests simulate remote failures with controlled

clients; invoice tests use real HTTP SOAP. These counts describe that saved run rather than a fresh execution of every later source change.

Table 6.6: Recorded Backend Integration Tests

Service	Tests	Main Coverage
User	3	Registration, hashing, login, roles and admin login.
Product	9	Reservations, concurrency, rollback, ownership, filtering and uploads.
Invoice	3	HTTP SOAP, WSDL, internal authentication and faults.
Order	8	Idempotency, recovery, cancellation, invoice failures and delivery.
Total	23	Zero reported failures or errors.

Saved evidence also records sixteen live gateway assertions and eleven demonstration-feature checks against the applications and CockroachDB. Frontend checks cover cart integrity, expired sessions, checkout retries, delivery fields, category results and photo metadata. Isolated tests reproduce edge cases, while live checks validate actual connectivity.

6.10 Running the Project

The project requires Java, Maven, a compatible Node.js runtime and access to the configured CockroachDB cluster. Local configuration stores private credentials. Run configuration once, reuse existing credentials and start the applications through the supplied scripts.

```
.\scripts\Build.ps1
.\scripts\Configure-Local.ps1
.\scripts\Initialize-Database.ps1
.\scripts\Start-Backend.ps1
.\scripts\Status-Backend.ps1
```

In a separate terminal, install frontend dependencies and start Angular. The gateway runs on port 8080 and the storefront on port 4200.

```
cd frontend
npm ci
npm start
```

The following commands populate sample products and verify the demonstration. Smoke tests create persistent verification records and should run against the demonstration environment.

```
.\scripts\Seed-Catalog.ps1
.\scripts\Smoke-Test.ps1
.\scripts\Verify-Demo-Features.ps1
# In the frontend directory
npm test
npm run build
```

CHAPTER 7
OUTPUT SCREENS AND RESULTS

7.1 Desktop Storefront

The retained desktop design capture shows the storefront header, introductory area and collection navigation. It documents an earlier interface state; the later catalog implementation supports search, filters, pagination and fifteen listings in each of five categories. The figure presents the opening portion of the saved full-page capture.

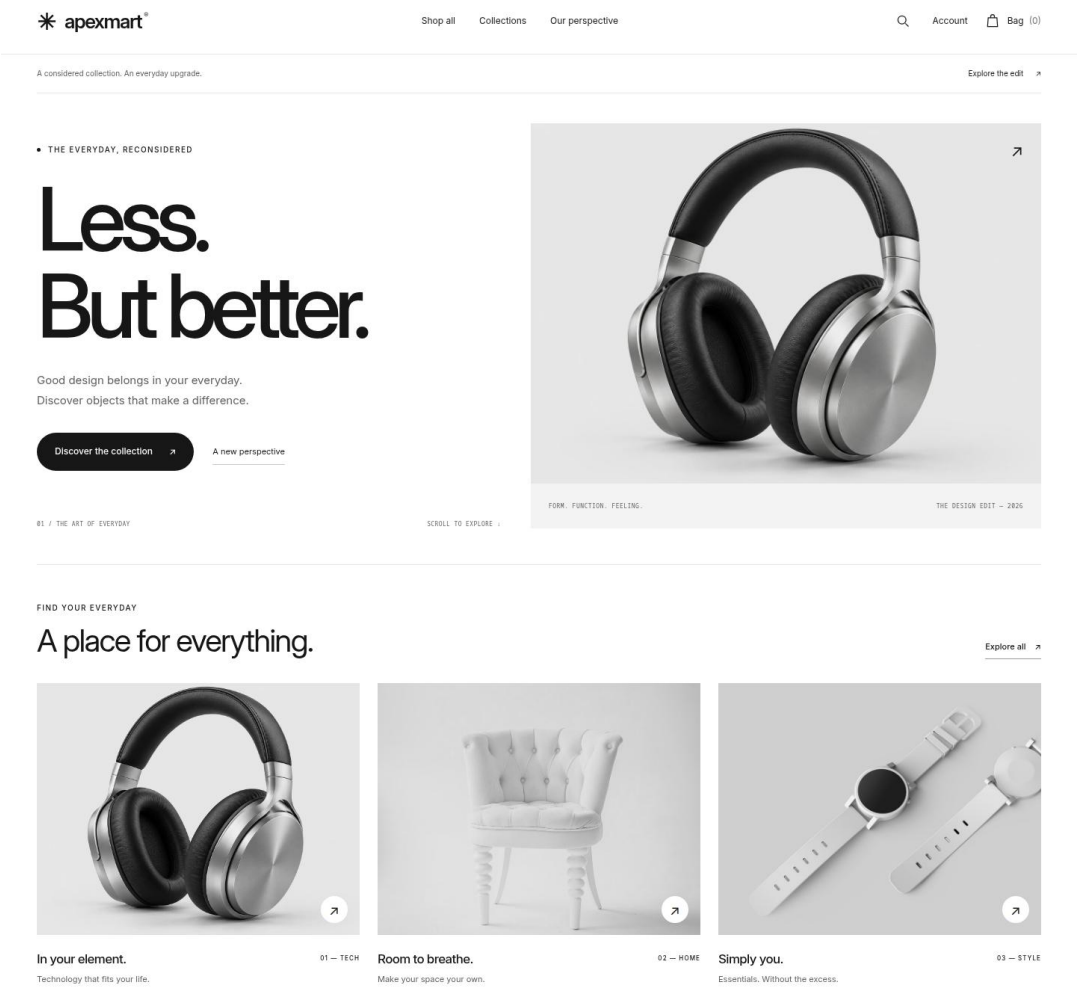


Figure 7.1: Desktop Storefront

7.2 Mobile Storefront

The mobile layout adapts navigation and catalog content to a narrow screen. The figure presents the opening portion of the retained mobile design capture. Separately, later saved browser verification records a 390-pixel viewport check with no horizontal page overflow.

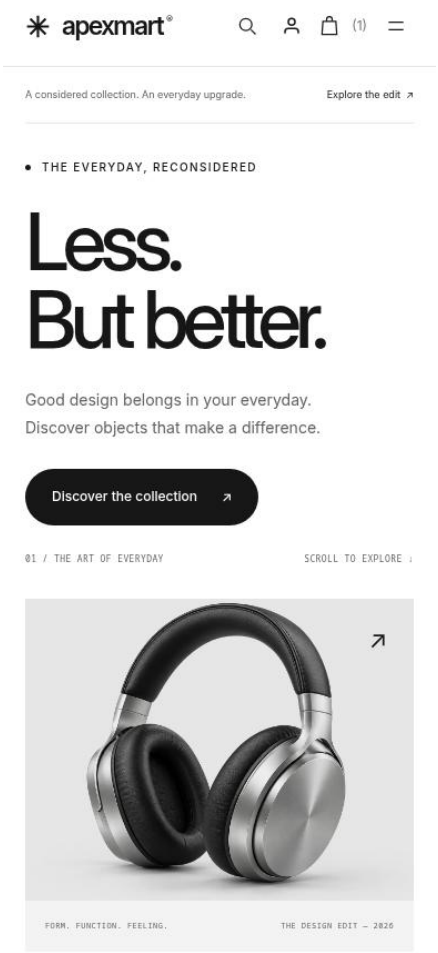


Figure 7.2: Mobile Storefront

7.3 Customer and Seller Workflows

Customer workflows connect browsing, cart validation, delivery details and order placement. Sellers manage owned listings and upload product photographs. Backend ownership checks remain effective when a caller bypasses interface navigation. Recorded API checks verify that customers cannot modify seller listings and sellers cannot use administrator product creation.

7.4 Checkout and Inventory Results

Live verification records confirmed checkout using server-calculated prices, replay returning the same order and stock deduction occurring once. Insufficient stock is rejected. Cancellation restores stock once, and repeated cancellation remains safe. These outcomes demonstrate consistent business state across retries.

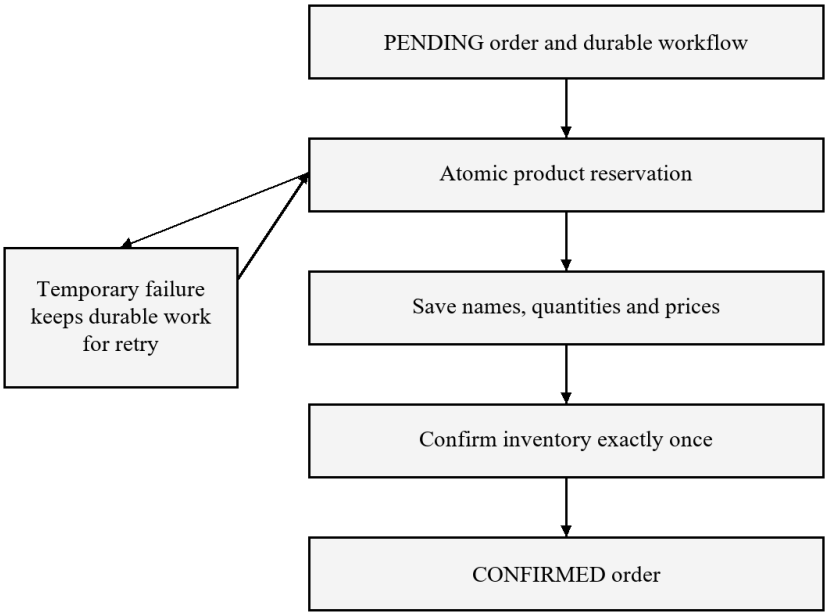


Figure 7.3: Recoverable Checkout Workflow

7.5 Invoice and Cancellation Results

The recorded gateway-to-SOAP flow generates an invoice and returns the same invoice on repetition. Starting invoice generation protects the order from cancellation. Automated invoice-outage scenarios verify that the guard remains durable when the dependency is unavailable. Retrying billing completes the work without restoring stock for an order whose billing has begun.

7.6 Validation and Access Results

Saved checks include invalid page sizes, insufficient stock, unauthorized order reads and invalid pincodes. Demonstration checks cover generated product codes, local photo persistence, public image retrieval, customer upload denial and unchanged checkout replay. Changing a payment method under an existing key is rejected.

7.7 Automated Test and Build Results

The saved testing document reports twenty-three backend integration tests with zero failures or errors, successful frontend regression checks and a successful Angular production build. Later storefront checks cover category rows, search, login persistence and mobile layout. These are completed recorded runs rather than newly executed tests for this report.

Table 7.1: Verification Summary

Verification Area	Recorded Outcome	Evidence
Backend integration	23 tests passed	Saved testing record.
Live gateway flow	16 assertions passed	Gateway verification JSON.
Demo features	11 live checks passed	Demo verification JSON.
Angular build and checks	Successful recorded runs	Frontend verification notes.
Desktop and mobile UI	Retained captures and browser checks	Storefront images and testing notes.

7.8 Observations and Limitations

The demonstration shows service separation and recovery-aware ordering. An intermittent cloud database timeout was observed during recorded browser checks; the services recovered and a later catalog retry succeeded. Availability therefore depends on the configured cluster and network. Saved tests do not establish production load capacity or behaviour under every possible failure.

Payments remain simulations, uploaded images remain local files and invoices are consolidated academic order bills. Fulfilment integration, tax calculation, refunds and statutory seller billing remain outside the implementation. These boundaries keep the project focused on the required API and microservice concepts.

CHAPTER 8
CONCLUSION

8.1 Summary

The E-Commerce Product and Order Management System demonstrates an academic flow from product discovery to recorded purchase and invoice retrieval. Angular provides the storefront, a gateway exposes a common API address, and four business services divide account, inventory, order and billing responsibilities. JPA persistence, migrations, validation and consistent errors support those responsibilities.

Recoverable checkout is a central part of the design. Idempotency prevents duplicate purchases, reservations protect stock, snapshots preserve history and durable workflows resume interrupted work. Cancellation compensates inventory once. Invoice generation uses a durable guard and repeatable SOAP contract. Each service commits only its own database transaction.

JWT authentication and backend authorization distinguish customer, seller and administrator operations. Recorded automated and live verification provides evidence for the core flows, and desktop and mobile captures demonstrate the storefront. The implementation meets its academic objective while maintaining clear boundaries around payment, fulfilment and production operations.

8.2 Future Enhancements

- Integrate a payment provider with verified callbacks, refunds and reconciliation.
- Add shipment tracking, fulfilment status and customer notifications.
- Introduce tax calculation, seller-specific invoices, PDF export and credit notes where required.
- Use separate database accounts and service identities, asymmetric JWT signing and token revocation.
- Move images to managed object storage with backup and lifecycle controls.
- Extend testing to process crashes, prolonged outages, database failover and production-scale concurrency.
- Add centralized metrics, tracing and alerts for unresolved workflows.
- Automate deployment and verification through continuous integration and managed secrets.

REFERENCES

Project implementation and verification sources used for this report:

[1] E-Commerce Product & Order Management. README.md. Overview, configuration and feature boundaries.

[2] Project architecture. docs/architecture.md. Service boundaries, inventory lifecycle and recovery.

[3] API guide. docs/api.md. REST requests, pagination and SOAP contract.

[4] Verification record. docs/testing.md, docs/verification.json and docs/demo-verification.json. Saved test evidence.

[5] Requirement coverage. docs/requirements.md. Mapping of academic requirements to implementation.

[6] Backend source and configuration. pom.xml; common, api-gateway, user-service, product-service, order-service and invoice-service modules.

[7] Database definitions. Versioned Flyway migrations in each service under src/main/resources/db/migration.

[8] Frontend source. frontend/package.json and frontend/src. Angular dependencies and client workflows.

[9] Storefront captures. docs/screenshots/storefront/desktop.png and mobile.png.